Module 3

Visible Surface Detection Methods: Depth Buffer, A-Buffer, Scan line

- Identifying those parts of a scene that are visible from a chosen viewing position.
- There are many approaches we can take to solve this problem, and numerous algorithms have been devised for efficient identification of visible objects for different types of applications.

- Some methods require more memory, some involve more processing time, and some apply only to special types of objects.
- Deciding upon a method for a particular application can depend on such factors as the complexity of the scene, type of objects to be displayed, available equipment, and whether static or animated displays are to be generated.
- The various algorithms are referred to as visible-surface detection methods. Sometimes these methods are also referred to as hidden-surface elimination methods

CLASSIFICATION OF VISIBLE-SURFACE DETECTION ALGORITHMS

- Q1.List out the different visible surface detection methods and explain any two visible surface detection methods
- Visible-surface detection algorithms are broadly classified according to whether they deal with object definitions directly or with their projected images.
- These two approaches are called object-space methods and image-space methods, respectively. **An object-space method** compare objects and parts of objects to each other within the scene definition to determine which surfaces, as a whole, we should label as visible. In an **image-space algorithm**, visibility is decided point by point at each pixel position on the projection plane. Most visible-surface algorithms use image-space methods, although object space methods can be used effectively to locate visible surfaces in some cases

- Line display algorithms, on the other hand, generally use object-space methods to identify visible lines in wireframe displays, but many image-space visible-surface algorithms can be adapted easily to visible-line detection.
- Although there are major differences in the basic approach taken by the various visible-surface detection algorithms, most use sorting and coherence methods to improve performance.
- Sorting is used to facilitate depth comparisons by ordering the individual surfaces in a scene according to their distance from the view plane. Coherence methods are used to take advantage of regularities in a scene.
- An individual scan line can be expected to contain intervals (runs) of constant pixel intensities, and scan-line patterns often change little from one line to the next.

DEPTH-BUFFER METHOD

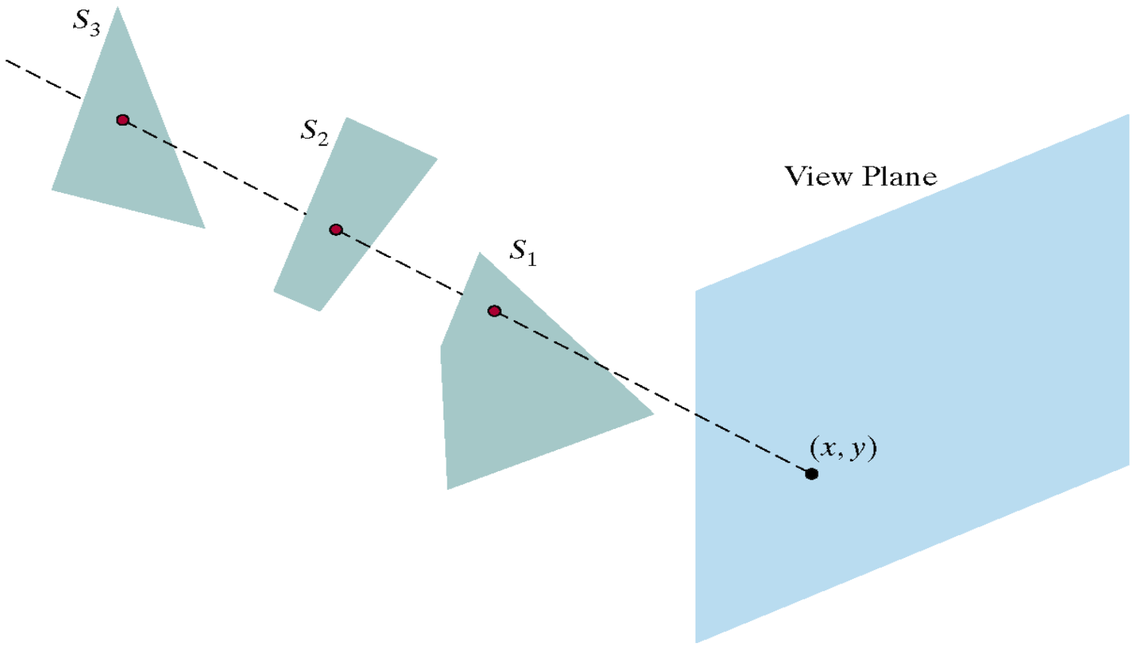
Q1.Explain the depth buffer method [5 marks , ktu dec 2017]

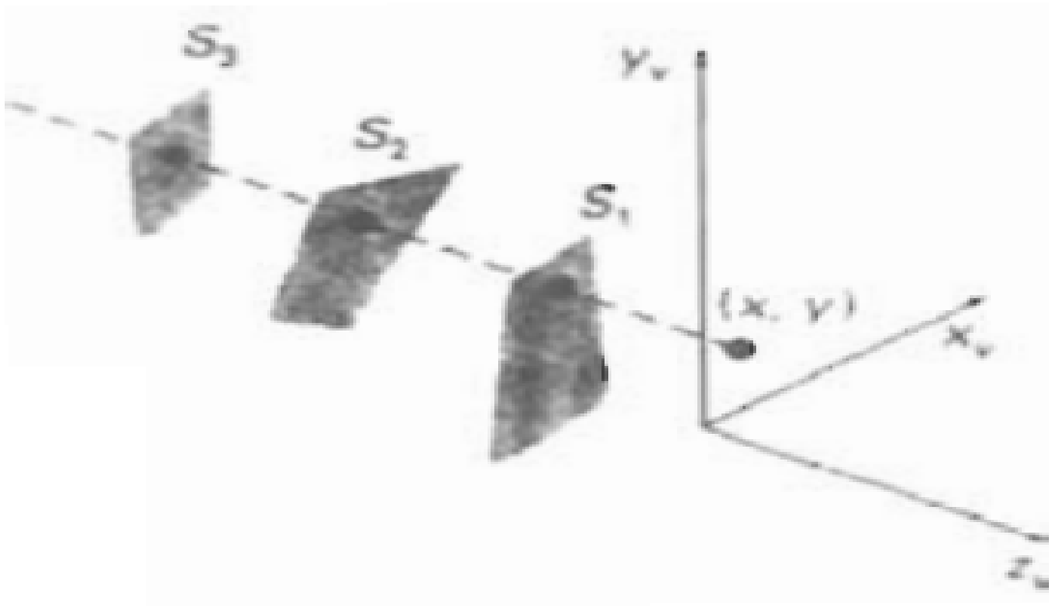
Q2.**Compare z buffer and A buffer algorithm? [6 marks, ktu dec 2017]**

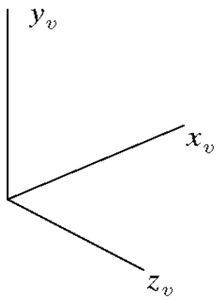
- A commonly used image-space approach to detecting visible surfaces is the depth-buffer method, which compares surface depths at each pixel position on the projection plane.
- This procedure is also referred to as the z-buffer method, since object depth is usually measured from the view plane along the z axis of a viewing system.
- Each surface of a scene is processed separately, one point at a time across the surface. The method is usually applied to scenes containing only polygon surfaces, because depth values can be computed very quickly and the method is easy to implement.
- But the method can be applied to nonplanar surfaces.

- With object descriptions converted to projection coordinates, each (x, y, z) position on a polygon surface corresponds to the orthographic projection point (x, y) on the view plane.
- Therefore, for each pixel position (x, y) on the view plane, object depths can be compared by comparing z values.
- Figure shows three surfaces at varying distances along the orthographic projection line from position (x, y) in a view plane taken as the x_v,y_v plane. Surface S_1, is closest at this position, so its surface intensity value at (x, y) is saved

- As implied by the name of this method, two buffer areas are required.
- A depth buffer is used to store depth values for each (x, y) position as surfaces are processed, and the refresh buffer stores the intensity values for each position.
- Initially, all positions in the depth buffer are set to 0 (minimum depth), and the refresh buffer is initialized to the background intensity.
- Each surface listed in the polygon tables is then processed, one scan line at a time, calculating the depth (z value) at each (x, y) pixel position. The calculated depth is compared to the value previously stored in the depth buffer at that position.
- If the calculated depth is greater than the value stored in the depth buffer, the new depth value is stored, and the surface intensity at that position is determined and in the same xy location in the refresh buffer.


S_3
S_2
S_1
View Plane
(x, y)


S_3
S_2
S_1
(x, y)


y_v
x_v
z_v

- We summarize the steps of a depth-buffer algorithm as follows:

1. Initialize the depth buffer and refresh buffer so that for all buffer positions (x, y),

$$\text{depth}(x, y) = 0, \qquad \text{refresh}(x, y) = I_{\text{backgnd}}$$

2. For each position on each polygon surface, compare depth values to previously stored values in the depth buffer to determine visibility.

   - Calculate the depth z for each (x, y) position on the polygon.
   - If $z > \text{depth}(x, y)$, then set

$$\text{depth}(x, y) = z, \qquad \text{refresh}(x, y) = I_{\text{surf}}(x,y)$$

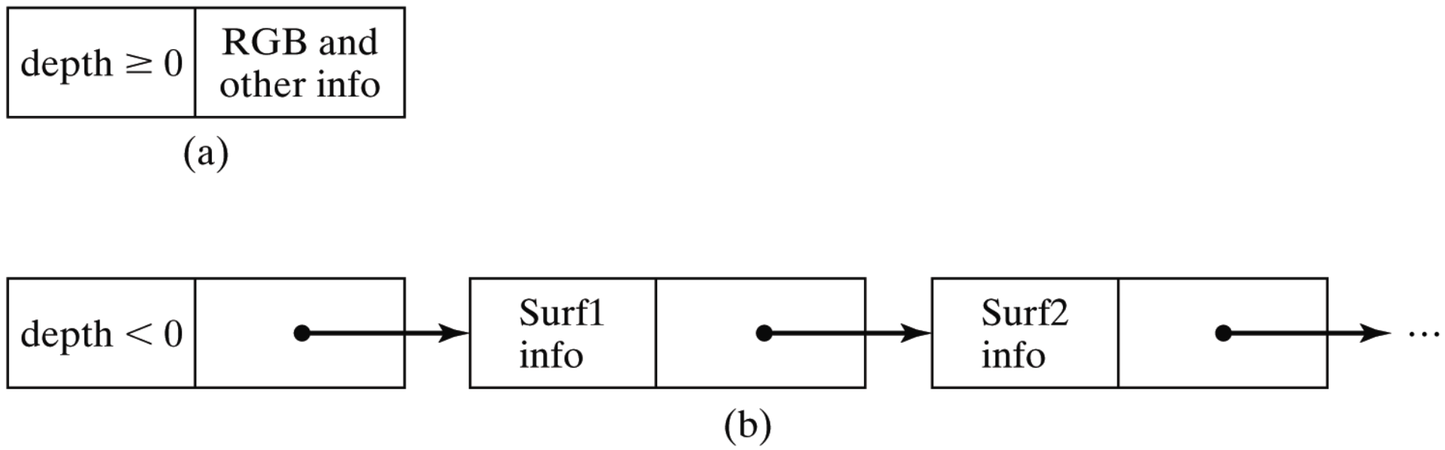
where I_{backgnd} is the value for the background intensity, and $I_{\text{surf}}(x,y)$ is the projected intensity value for the surface at pixel position (x,y). After all surfaces have been processed, the depth buffer contains depth values for the visible surfaces and the refresh buffer contains the corresponding intensity values for those surfaces.

- Depth values for a surface position (x, y) are calculated from the plane equation for each surface

$$z = \frac{-Ax - By - D}{C}$$

A-Buffer Method

- The A-buffer method is an extension of the depth-buffer method (at the other end of the alphabet from "z-buffer", where z represents depth).
- The A buffer method represents an antialiased, area-averaged, accumulation-buffer method .
- A drawback of the depth-buffer method is that it can only find one visible surface at each pixel position.
- In other words, it deals only with opaque surfaces and cannot accumulate intensity values for more than one surface, as is necessary if transparent surfaces are to be displayed.
- The A-buffer method expands the depth buffer so that each position in the buffer can reference a linked list of surfaces. Thus, more than one surface intensity can be taken into consideration at each pixel position, and object edges can be antialiased



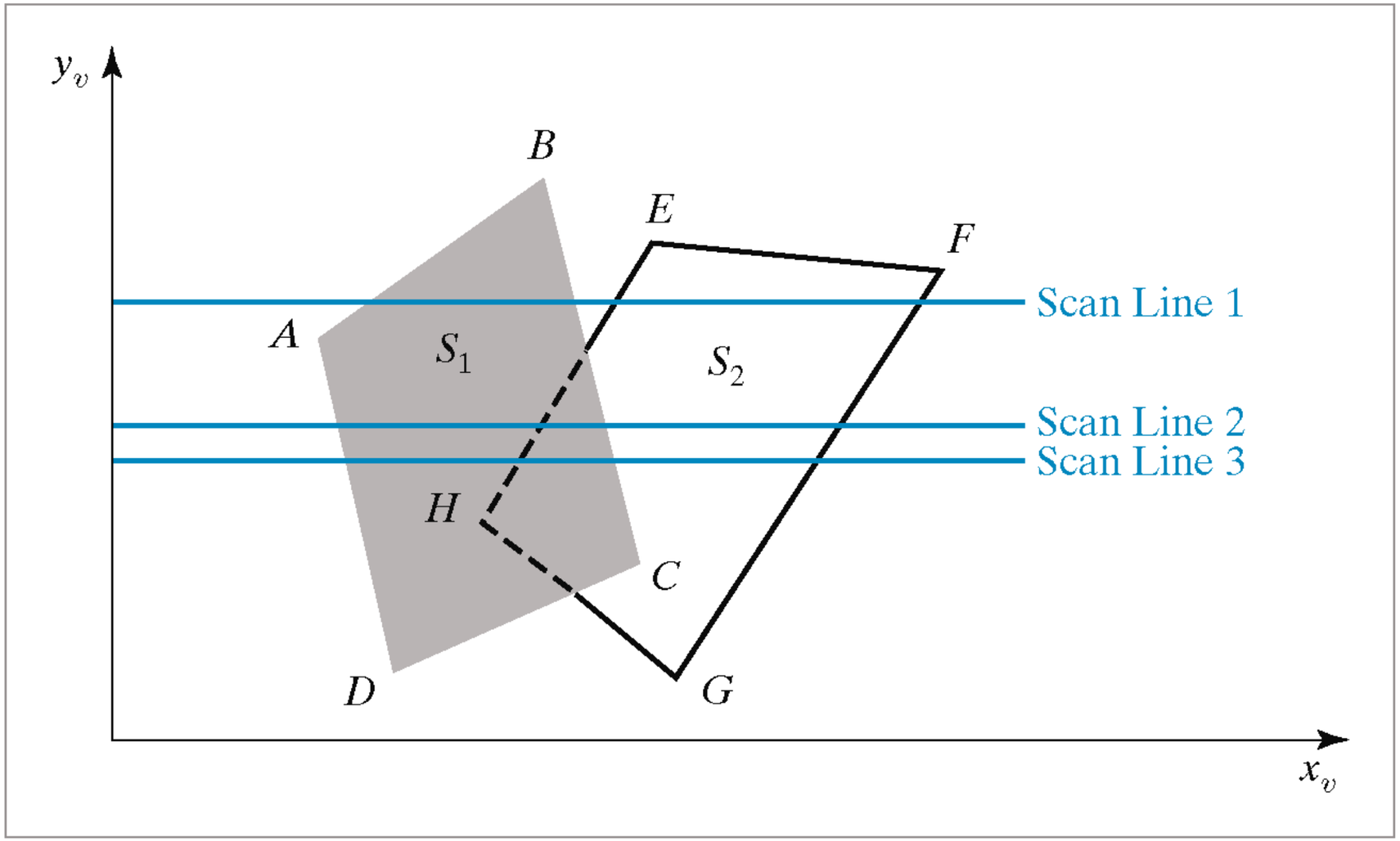
- Each position in the A-buffer has two fields:
- depth field - stores a positive or negative real number
- intensity field - stores surface-intensity information or a pointer value.

- If the depth field is positive, the number stored at that position is the depth of a single surface overlapping the corresponding pixel area.
- The intensity field then stores the RGB components of the surface color at that point and the percent of pixel coverage,
- If the depth field is negative, this indicates multiple-surface contributions to the pixel intensity.
- The intensity field then stores a pointer to a linked list of surface data.
- Data for each surface in the linked list includes
    - RGB intensity components
    - opacity parameter (percent of transparency)
    - depth percent of area

Scan-Line Method

1. Explain Scan line algorithm [5 marks, ktu dec 2017]

- An image space method for identifying visible surfaces
- Computes and compares depth values along the various scan-lines for a scene
- This image space method for removing hidden surfaces is an extension of the scan-line algorithm for filling polygon interiors.
- Instead of filling just one surface, we now deal with multiple surfaces. As each scan line is processed, all polygon surfaces intersecting that line are examined to determine which are visible.
- Across each scan line, depth calculations are made for each overlapping surface to determine which is nearest to the view plane. When the visible surface has been determined, the intensity value for that position is entered into the refresh buffer


y_v
B
E
F
Scan Line 1
A
S_1
S_2
Scan Line 2
Scan Line 3
H
C
D
G
x_v

- We assume that tables are-set up for the various surfaces, as discussed, which include both an edge table and a polygon table.
- The edge table contains coordinate endpoints for each line in-the scene, the inverse slope of each line, and pointers into the polygon table to identify the surfaces bounded by each line.
- The polygon table contains coefficients of the plane equation for each sur- face, intensity information for the surfaces, and possibly pointers into the edge Scan-Line Method table.
- To facilitate the search for surfaces scan line, we can set up an active list of edges from information in the edge table.
- This active list will contain only edges that cross the current scan line, sorted in order of increasing x.
- In addition, we define a flag for each surface that is set on or off to indicate whether a position along a scan line is inside or outside of the surface.
- Scan lines are processed from left to right. At the leftmost boundary of a surface, the surface flag is turned on; and at the rightmost boundary, it is turned of

- Illustrates the scan-line method for locating visible portions of surfaces for pixel positions along the line. The active list for line 1 contains information from the edge table for edges AB, BC, EH, and FG. For positions along this scan line between edges AB and BC, only the flag for surface S1 is on.
- Therefore, no depth calculations are necessary, and intensity information for surface S1 is entered from the polygon table into the refresh buffer. Similarly, between edges EH and FG, only the flag for surface S2 is on. NO other positions along scan line 1 intersect surfaces, so the intensity values in the other areas are set to the background intensity.
- The background intensity can be loaded throughout the buffer in an initialization routine

- For scan lines 2 and 3, the active edge list contains edges AD, EH, BC, and FG. Along scan line 2 from edge AD to edge EH, only the flag for surface S1 is on. But between edges EH and BC, the flags for both surfaces are on. In this interval, depth calculations must be made using the plane coefficients for the two surfaces. For this example, the depth of surface SI is assumed to be less than that of S2, so intensities for surface S1 are loaded into the refresh buffer until boundary BC is encountered. Then the flag for surface SI goes off, and intensities for surface S2 are stored until edge FG is passed

- Scan line 3 has the same active list of edges as scan line 2. Since no changes have occurred in line intersections, it is unnecessary again to make depth calculations between edges EH and BC.